💙

Week6_예습과제_김재희

1. Introduction
    - 논문이 다루는 분야
    - 해당 task에서 기존 연구 한계점
    - 논문의 contributions
2. Related Work
3. 제안 방법론
    - Main Idea
    - Contribution
4. 실험 및 결과
    - Dataset
    - Baseline
    - 결과
5. 결론 (배운점)

1. Introduction

> 💡 논문에서 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
> 논문에서 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

- 자연어 처리

해당 task에서 기존 연구 한계점

- 기존 모델들은 pre-trained된 모델의 전체 파라미터를 전부 fine-tuning해서 deploy 할 때 엄청난 inference latency를 겪었고, 메모리 사용량 또한 많았다.

논문의 contributions

- pre-trained된 모델의 핵심 공용 파라미터는 유지하고, 새로운 작은 배열들인 A와 B만 새로 파인튜닝한다. 이를 통해 파라미터 양은 1,000배가량 줄일 수 있었고 메모리 사용량은 3배가량 줄일 수 있었다.

2. Related Work

> 💡 Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
> 제안 방법의 차별성을 어떻게 표현하고 있는가

- **Adapter 계열**: Transformer 블록 사이에 작은 adapter 레이어를 추가. 파라미터는 적지만 순차 연산이라 **inference latency 증가.**
- **Prompt/Prefix tuning**: 입력 쪽에 학습 가능한 토큰을 붙임. 최적화가 어렵고 성능이 불안정하며 시퀀스 길이를 먹어서 **실제 입력에 쓸 수 있는 길이가 줄어듦**.
- **Intrinsic dimension 연구**: 과파라미터화된 모델도 저차원 부공간에서 학습 가능하다는 관찰. LoRA의 이론적 동기.
- **LoRA의 차별성**: 저랭크 분해 행렬을 학습하고 배포 시 원본 가중치에 **merge**할 수 있어서 **추가 latency가 0**. 입력 길이도 안 줄어듦.

3. 제안 방법론

> 💡 Introduction에서 언급된 내용과 동일하게 작성되어 있는가
> Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
> 제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

- 사전학습 가중치 W_0는 freeze하고, 업데이트량 ΔW를 저랭크로 분해해서 학습: **$W = W_0 + \Delta W = W_0 + BA$**
  - $B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times k)}$, $r \ll \min(d, k)$
  - forward: $h = W_0x + BAx$ (스케일링 α/r 적용)
- 초기화: **A는 가우시안, B는 0** → 학습 시작 시 $\Delta W = 0$이라 원래 모델과 동일하게 출발
- 적용 위치: Transformer attention의 **W_q, W_v**에 적용
- 가정: fine-tuning 중 가중치 변화량은 intrinsic rank가 낮다

Contribution

장점	근거
학습 파라미터 ↓	GPT-3 175B에서 약 10,000배 감소, 체크포인트 350GB → 35MB 수준
메모리 ↓	학습 시 VRAM 약 3배 절약
inference latency 0	배포 시 W_0 + BA를 미리 합쳐서 일반 모델처럼 사용
태스크 전환 쉬움	A, B만 갈아끼우면 됨

구현 가능성: 수식이 단순해서 linear layer 옆에 병렬 low-rank 경로(A→B)만 추가하면 됨. 공식 구현은 `loralib,` 지금은 HuggingFace PEFT로 바로 씀.

4. 실험 및 결과

💡 Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

- RoBERTa / DeBERTa: GLUE
- GPT-2: E2E NLG Challenge, WebNLG, DART
- GPT-3 175B: WikiSQL, MNLI, SAMSum

Baseline

- Full Fine-tuning (FT)
- BitFit (bias만 학습)
- Prefix-embedding / Prefix-layer tuning
- Adapter 계열 (Adapter^H, ^L, ^P, ^D)

결과

- RoBERTa, DeBERTa, GPT-2: 학습 파라미터는 훨씬 적은데 **FT와 비슷하거나 더 높은 성능**
- GPT-3 175B: WikiSQL, MNLI, SAMSum에서 FT와 동등하거나 약간 우위. Prefix 계열은 학습 파라미터를 늘려도 성능이 불안정했음
- Ablation:
  - r이 매우 작아도(r=1~4) 충분히 잘 동작 → 저랭크 가정 뒷받침

- 같은 파라미터 예산이면 W_q 하나에 큰 r보다 **W_q, W_v 둘 다에 작은 r**이 더 좋음
- ΔW는 W의 top singular direction보다 **W에서 강조되지 않던 방향을 증폭**하는 경향

5. 결론 (배운점)



연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

- **의의**: 성능 손실 없이 파인튜닝 비용(학습 메모리, 저장, 배포)을 크게 낮춤. 이후 PEFT의 사실상 표준이 됨 (QLoRA 등으로 확장)
- **한계점**
  - 서로 다른 A, B를 쓰는 입력을 **한 배치에 섞어서** 처리하려면, merge하면 안 되고 별도 계산이 필요함 (latency tradeoff)
  - 어떤 가중치에 적용할지(W_q, W_v 등)가 경험적 선택이라 이론적 가이드가 약함
  - rank r 선택도 휴리스틱